



How to Build Data Systems Fall 2026, Lecture 1: What We'll Build, How We'll Work

Agenda

Welcome

- Course overview
- AI policy & working with AI
- How much work is this?
- Architecture

Coffee break: Let's form teams

Workflow

- The engineering toolchain
- The GitHub lifecycle
- Logging from day one
- Dogfooding as design philosophy

This week's exercise and reading

Martin Hentschel



IT UNIVERSITY OF COPENHAGEN

Associate Professor



Principal Software Engineer



Software Engineer



BSc, MSc, PhD in Computer Science

Omar Khan

IT UNIVERSITY OF COPENHAGEN



IT UNIVERSITY OF COPENHAGEN

IT UNIVERSITY OF COPENHAGEN

Assistant Professor

Postdoctoral Researcher

Postdoctoral Researcher

BSc, MSc, PhD in Computer Science



Teaching Assistant

Alperen Aydın

GitHub username **alqeren1**



IT UNIVERSITY OF COPENHAGEN

Studying MSc in Software Design



BSc of Architecture

IT UNIVERSITY OF COPENHAGEN

Intended Learning Outcomes

1. Gain proficiency in using tools for unit testing, end-to-end testing, and continuous integration/continuous deployment (CI/CD) pipelines
2. Reflect on data storage formats, such as Apache Parquet, CSV, and binary, and be able to choose the appropriate format for different use cases
3. Parse, optimize, and execute SQL queries based on statistics, including join ordering and pruning
4. Apply different query operator execution techniques given changing data distributions and scaling factors
5. Analyze and address scalability issues, including memory pressure and CPU overload, to handle large datasets efficiently
6. Critically evaluate and reflect on different approaches to database system design and implementation

We will build a database system from scratch!

Demo

(create a table, copy CSV file, run a select query, analyze logs)

Why Build From Scratch?

Lots of decision making and choosing from alternatives

- Forces you to understand and discuss trade-offs
- Things you implement = things you understand better
- Make informed decisions in your job later
- But...does AI change that?

Why An Analytical Engine?

We are building a small OLAP engine, including:

- Storage with immutable partitions / files
- Transactions, but single-statement
- Joins

We are not building, or only optionally building:

- Buffer pools, WAL, B-trees, In-Place updates, RBAC, Encryption, Group-bys, Window functions, UDFs, ...
- Main reason: We want to finish this semester

Course Outline

Date	Lecture & Exercise Topics	Project
Aug 27	What we'll build, how we'll work	P1 release
Sep 3	Storing data: partitions, statistics, catalog	
Sep 10	From text to tree: parsing SQL	
Sep 17	From tree to rows: executing queries	
Sep 24	<i>No class and exercise</i>	
Oct 1	Dogfooding: logging and production testing	P1 hand in, P2 release
Oct 8	Benchmarking: design, run, measure, report	
Oct 15	<i>No class and exercise, Fall holiday</i>	
Oct 22	Light-weight transactions I: copy-on-write	
Oct 29	Light-weight transactions II: conflict resolution	
Nov 5	Testing data systems: concurrency and parallelism	P2 hand in, P3 release
Nov 12	Joins: algorithms and multi-way plans	
Nov 19	The optimizer: cardinality estimation, join ordering	
Nov 26	What we skipped, where engines go	
Dec 3	<i>No class and exercise</i>	P3 hand in

Project & Exam

Either submit three reports based on each project or one report that bundles all three.

Official submission date is January 11, 2027.

Recommended: submit individual reports by the dates in the previous slide.

Exam type:

D: Submission of written work with following oral, External (7-point scale)

Exam variation:

D2M: Submission for groups or individual with following oral exam supplemented by the submission.

Exam submission description:

Students submit a report based on three homework assignments and link to Github source code repository. This report is submitted for the exam.

Group submission:

Group, Recommended group size: 2 students.

Exam duration per student for the oral exam:

20 minutes

Group exam form:

Individual exam : Individual student presentation followed by an individual dialogue. The student is examined while the rest of the group is outside the room.

Agenda

Welcome

- Course overview
- **AI policy & working with AI**
- How much work is this?
- Architecture

Coffee break: Let's form teams

Workflow

- The engineering toolchain
- The GitHub lifecycle
- Logging from day one
- Dogfooding as design philosophy

This week's exercise and reading



Claude



GitHub Copilot

During project work: Absolutely!

During Exam: No!



AI is allowed and encouraged

- Industry standard in 2026
- Too much work without it

Choose any coding agent you want

- AI attribution in PRs and in your reports

But

- You “own” every line of code: must be able to explain it
- Code & tests(!) can be slop, verify!
- For me:
exercise implementation: 15-20mins using Claude Opus 5
understanding code & playing around with it: minimum 2 hours

Working With AI

You write down design decisions beforehand,
sometimes as explicitly required in the exercises

AI writes the code and tests

You validate the code through PR reviews

- If you cannot explain it, don't merge it!
- “Tests can be slop” – Rushabh Doshi, 2026

You can ask AI questions about the code

You reflect on your use of AI in the project reports

Agenda

Welcome

- Course overview
- AI policy & working with AI
- How much work is this?
- Architecture

Coffee break: Let's form teams

Workflow

- The engineering toolchain
- The GitHub lifecycle
- Logging from day one
- Dogfooding as design philosophy

This week's exercise and reading

Course Workload

One ECTS corresponds to approximately 27.5 hours of work for the students, in line with the official European Credit Transfer System.

A 7.5 ECTS course therefore represents around 206 hours of activity in total.

These hours include contact hours (teaching and supervision), individual study time, group work and exam activities, corresponding to an average of approximately **10 hours per week** from the first day of teaching to the last day of the exam period.

In practice there will be fluctuations, as there often is more work required during teaching weeks and when working on projects.

Student Activity Budget

Estimated distribution of learning activities for the typical student

- Preparation for lectures and exercises: 10%
- Lectures: 25%
- **Assignments: 60%**
- Exam with preparation: 5%

Lines of code of my sample implementation

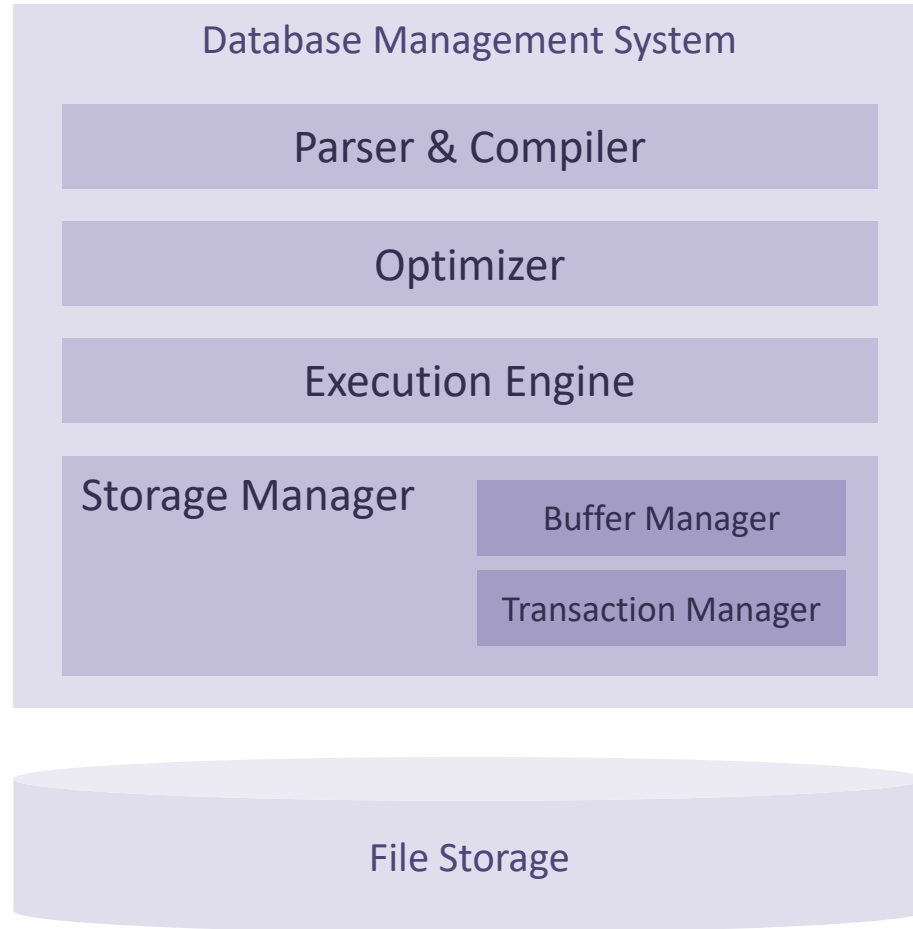
After	Java main	Java test	Java total	With grammar, scripts, build
Part 1 (v0.6)	1,296	1,079	2,375	2,827
Part 2 (v0.8)	1,747	1,669	3,416	3,872
Part 3 (v1.0)	2,653	2,711	5,364	6,113

- Part 2 is the smallest increment (about 1,050 lines) because copy-on-write reuses the Part 1 structures; Part 3 is the largest (about 2,240 lines) because joins, statistics, and the optimizer each need new operators and new tests.
- My finished engine is about 5,400 lines of Java in 69 files.

Recap: Architecture of a Database*

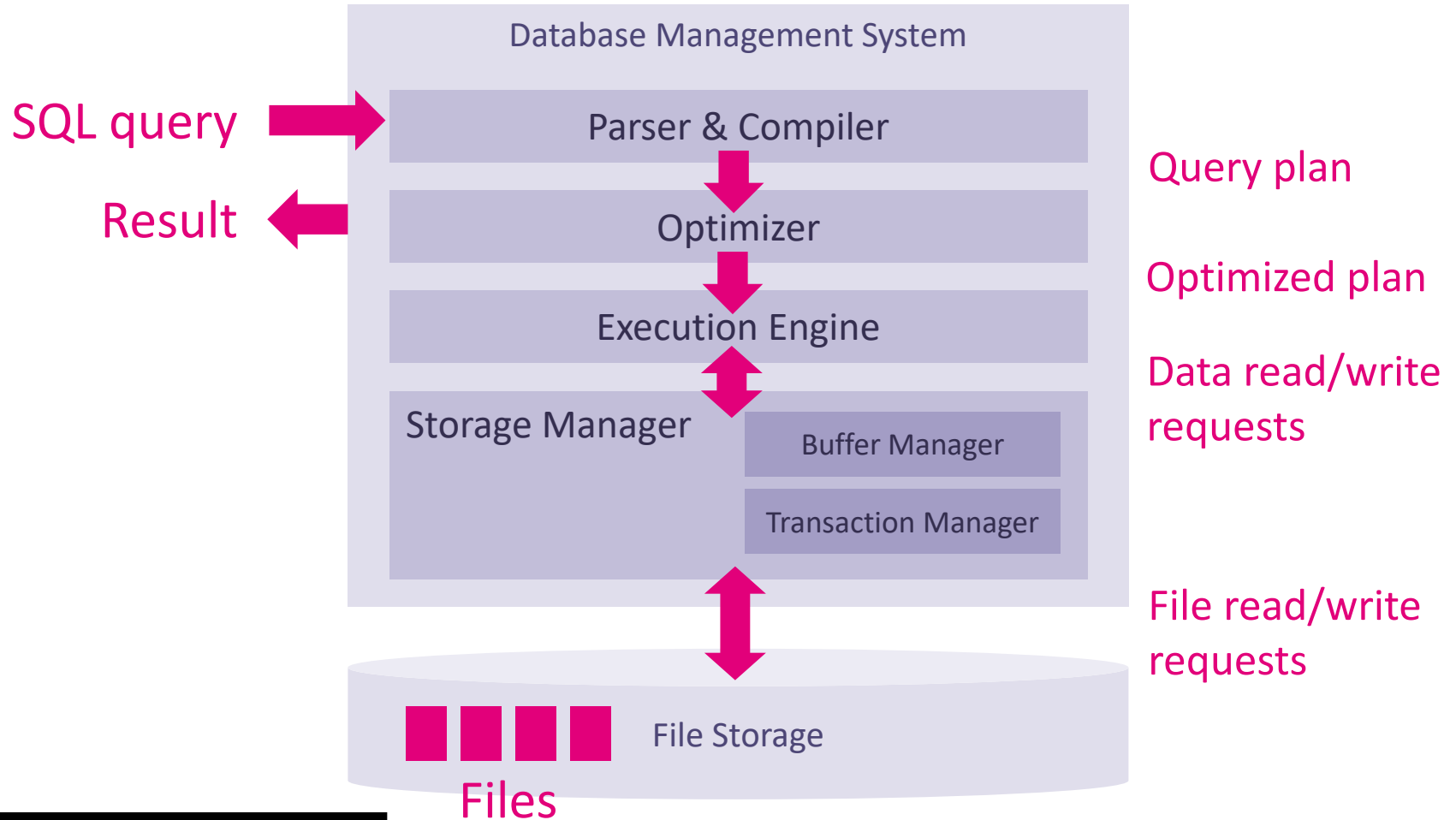
Introduction to Database Systems,
Lecture 9

Important
components
of a database
system

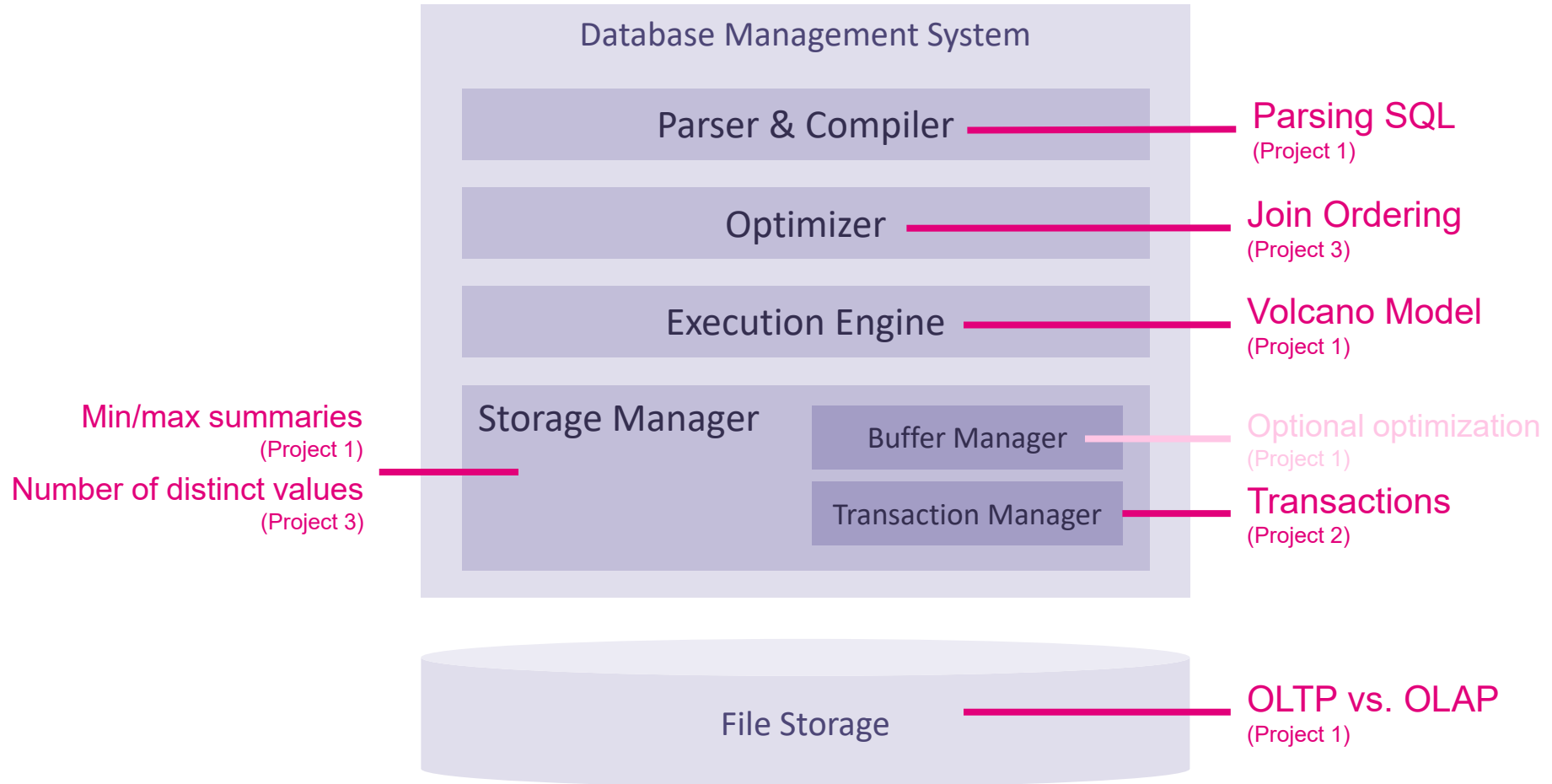


*Only a few important components shown. Many more components exist in practice, varying by database system.

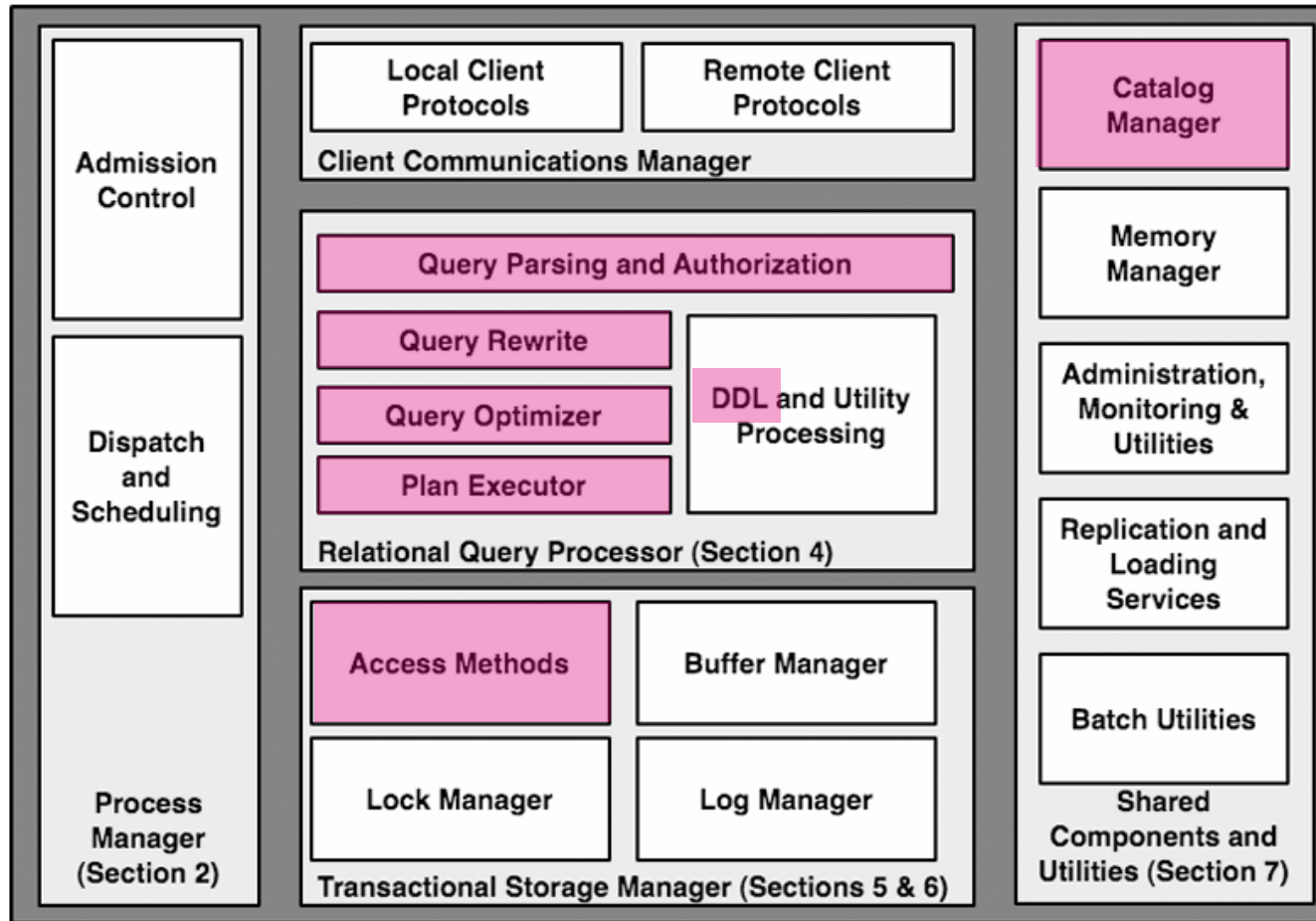
Flow of Information



Components Covered in this Course



Main Components



DuckDB as a Reference System



Every command your engine supports runs unchanged in DuckDB

- You already know DuckDB from Intro DB
- Your engine speaks a real SQL dialect, not a course-only language
- Built-in correctness oracle: run the same script through both, diff the results ("differential testing")

From week 4 on: engine output vs. `duckdb -csv -noheader`, then diff

Coffee Break

Let's form teams

2-person teams recommended

Think of a team name



Agenda

Welcome

- Course overview
- AI policy & working with AI
- How much work is this?
- Architecture

Coffee break: Let's form teams

Workflow

- The engineering toolchain
- The GitHub lifecycle
- Logging from day one
- Dogfooding as design philosophy

This week's exercise and reading

Tools used in this course

- Java
- Maven build tool
- JUnit test suite
- GitHub

These are given, you cannot change this

- Comparability across groups
- Simulates using existing tool set in a company

Latest Java long-term support (LTS) = Java 25

- Allowed and encouraged to use its language features, e.g., sealed interfaces and switch expressions

```
public sealed interface Statement
    permits CreateTableStatement, CopyStatement, SelectStatement, DeleteStatement { }

/** Renders a statement back to SQL text that parses to an equal statement. */
public String print(Statement statement) {
    return switch (statement) {
        case CreateTableStatement s ->
            "CREATE TABLE " + s.tableName() + " (" + columnList(s.columns()) + ");";
        case CopyStatement s ->
            "COPY " + s.tableName() + " FROM " + quote(s.csvFilePath()) + ";";
        case SelectStatement s ->
            "SELECT * FROM " + String.join(delimiter: ", ", s.tableNames()) + whereClause(s) + ";";
        case DeleteStatement s ->
            "DELETE FROM " + s.tableName() + whereClause(s.where()) + ";";
    };
}
```

Vs. Visitor Pattern

Common in industry

(because it was the best pattern before sealed classes)

1. Visitor interface
2. Every class/record gets an accept method
3. Visitors turn functions into a standalone class (next slide)

```
public interface StatementVisitor<R> {  
    R visit(CreateTableStatement s);  
    R visit(CopyStatement s);  
    R visit(SelectStatement s);  
    R visit>DeleteStatement s);  
}
```

```
public sealed interface Statement  
    permits CreateTableStatement, CopyStatement, SelectStatement, DeleteStatement;  
    // Every class in the hierarchy must agree to accept visitors  
    <R> R accept(StatementVisitor<R> visitor);  
}  
  
public record CreateTableStatement(String tableName, List<Column> columns) implements Statement  
    @Override  
    public <R> R accept(StatementVisitor<R> visitor) {  
        return visitor.visit(this); // Double-dispatch happens here  
    }  
}  
  
public record CopyStatement(String tableName, String csvFilePath) implements Statement  
    @Override  
    public <R> R accept(StatementVisitor<R> visitor) {  
        return visitor.visit(this);  
    }  
}  
  
public record SelectStatement(List<String> tableNames, Where where) implements Statement  
    @Override  
    public <R> R accept(StatementVisitor<R> visitor) {  
        return visitor.visit(this);  
    }  
}
```

3. Visitors in own class

```
public class StatementPrinterVisitor implements StatementVisitor<String> {

    public String print(Statement statement) {
        // You kickoff the double-dispatch loop here
        return statement.accept(this);
    }

    @Override
    public String visit(CreateTableStatement s) {
        return "CREATE TABLE " + s.tableName() + " (" + columnList(s.columns()) + ";";
    }

    @Override
    public String visit(CopyStatement s) {
        return "COPY " + s.tableName() + " FROM " + quote(s.csvFilePath()) + ";";
    }

    @Override
    public String visit(SelectStatement s) {
        return "SELECT * FROM " + String.join(", ", s.tableNames()) + whereClause(s.where());
    }

    @Override
    public String visit>DeleteStatement s) {
        return "DELETE FROM " + s.tableName() + whereClause(s.where()) + ";";
    }
}
```

Sealed Interfaces + Switch vs. Visitor Pattern

Feature	Sealed Interfaces + Switch	Visitor Pattern
Boilerplate	Low. Records declare the data; a switch declares the behavior. No <code>accept()</code> , no visitor interface.	High. A <code>Visitor<R></code> interface, an <code>accept()</code> override in every node, and a concrete class per operation.
Adding a new node type	Easy. Every exhaustive switch stops compiling, pointing at exactly the passes that need updating.	One manual step. You edit the visitor interface first, after which implementors also fail to compile.
Adding a new operation	Easy. A new method with a switch in it.	Easy. A new class implementing <code>Visitor</code> . (This is the problem <code>Visitor</code> was invented to solve.)
Dispatch expressiveness	Rich. Nested record patterns match several levels at once. <code>when</code> guards split one node type by condition. Constant folding is a single <code>case</code> .	One node at a time. Multi-level or conditional matching means hand-written <code>instanceof</code> chains inside a <code>visit</code> method.
Extension outside your module	Closed by design. <code>sealed</code> forbids new node types elsewhere. The right default for an AST (Lecture 3).	Open. Third parties can add node types, at the cost of breaking every existing visitor.

We are using Maven as build tool for our project

- Common in industry
- Typically you cannot choose the build tool

How It Works

- **Predefined structure:** Maven expects files in specific places
- **Standard layout:** Source code belongs in `src/main/java`
- **Test layout:** Test code belongs in `src/test/java`
- **Output layout:** Compiled files always go to the `target` folder

Project configuration file (`pom.xml`) “only” needs to list your project's name and its dependencies.

Minimal pom.xml

Components

- **modelVersion**: Defines the POM structure version (always 4.0.0 for Maven 2 and 3)
- **groupId**: The organization or package name (usually reversed domain name)
- **artifactId**: The unique name of your specific project
- **version**: The current version of your project

This file is enough to compile code in
src/main/java and run tests in
src/test/java without any further setup.

```
<project xmlns="http://apache.org"
          xmlns:xsi="http://w3.org"
          xsi:schemaLocation="http://apache

          <modelVersion>4.0.0</modelVersion>

          <groupId>dk.itu.datasys</groupId>
          <artifactId>engine</artifactId>
          <version>0.1.0</version>
</project>
```

pom.xml in Practice

Long list of:

Dependencies

- Each external library requires a separate XML block including groupId, artifactId, version

Plugins

- Tasks like source code generation of Lexer and Parser, packaging, running

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/maven-v4_0_0.xsd"
  <modelVersion>4.0.0</modelVersion>

  <groupId>dk.itu.datasys</groupId>
  <artifactId>engine</artifactId>
  <version>0.1.0</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.release>25</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <log4j.version>2.26.1</log4j.version>
    <surefire.version>3.5.6</surefire.version>
    <antlr.version>4.13.2</antlr.version>
  </properties>

  <dependencies>
    <!-- Logging API: your code compiles only against slf4j-api -->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>2.0.18</version>
    </dependency>
    <!-- Logging backend: Log4j2, needed at runtime only -->
    <dependency>
      <groupId>org.apache.logging.log4j</groupId>
      <artifactId>log4j-slf4j-impl</artifactId>
      <version>${log4j.version}</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.apache.logging.log4j</groupId>
      <artifactId>log4j-core</artifactId>
      <version>${log4j.version}</version>
      <scope>runtime</scope>
    </dependency>
    <!-- JSON for the catalog file -->
    <dependency>
      <groupId>tools.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>3.2.1</version>
    </dependency>
    <!-- Runtime half of ANTLR; the lexer and parser -->
    <dependency>
      <groupId>org.antlr</groupId>
      <artifactId>antlr4-runtime</artifactId>
      <version>${antlr.version}</version>
    </dependency>
    <!-- Testing -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>6.1.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

```
<build>
  <!-- One name for the artifact, so the engine script can name the finalName>engine</finalName>
  <plugins>
    <!-- Compiles with the Java release set above. Pinned because Maven picks by default is too old to understand 'maven.compiler.release' -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.15.0</version>
    </plugin>
    <!-- Turns src/main/antlr4/**/Sql.g4 into a lexer, a parser and target/generated-sources/antlr4, which joins the compiler -->
    <plugin>
      <groupId>org.antlr</groupId>
      <artifactId>antlr4-maven-plugin</artifactId>
      <version>${antlr.version}</version>
      <configuration>
        <!-- Off by default; without it there is no SqlBaseVisitor -->
        <visitor>true</visitor>
      </configuration>
      <executions>
        <execution>
          <goals><goal>antlr4</goal></goals>
        </execution>
      </executions>
    </plugin>
    <!-- Runs the unit tests (*Test) during 'mvn test' / 'mvn package' -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>${surefire.version}</version>
    </plugin>
    <!-- Runs the integration tests (*IT) during 'mvn verify' -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-failsafe-plugin</artifactId>
      <version>${surefire.version}</version>
      <executions>
        <execution>
          <goals>
            <goal>integration-test</goal>
            <goal>verify</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
    <!-- Lets 'mvn exec:java' start the engine -->
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.6.3</version>
      <configuration>
        <mainClass>dk.itu.datasys.Engine</mainClass>
      </configuration>
    </plugin>
    <!-- Writes the classes and all dependencies into one runnable transformer records the main class; the services transformer which SLF4J finds no Log4j2 backend in the merged JAR -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-shade-plugin</artifactId>
      <version>3.6.2</version>
      <executions>
        <execution>
```

Useful Maven commands

Command	What it does
<code>mvn clean</code>	Deletes the <code>target/</code> directory and removes previous build output
<code>mvn compile</code>	Compiles the main source code
<code>mvn test</code>	Compiles and runs the unit tests
<code>mvn package</code>	Compiles, tests, and packages the application into a JAR/WAR
<code>mvn verify</code>	Runs all checks needed to verify the project is valid, including tests and integration checks configured in the build. <code>mvn -B verify</code> is what GitHub CI does (non-interactive mode)
<code>mvn exec:java</code>	Executes a Java class using the Maven Exec plugin

Phases are cumulative

- package implies test, verify implies package
- Running a phase runs everything before it

-B is batch mode: no colors, no download progress bars, readable CI logs

- if `mvn -B verify` passes on your machine, CI passes

Maven has no start goal

- Building is Maven's job running is not
- The exec plugin fills that gap, which is why it is in the POM

@Test marks a test method; surefire finds every class named *Test.java on its own

Assertions: assertEquals, assertTrue, assertEquals, assertEquals, assertEquals

- The expected value comes first: assertEquals(expected, actual)

Course convention: one unit test file per class. SqlPrinter.java has SqlPrinterTest.java.

```
@Test
void printPredicateColumnOperatorConstant() {
    assertEquals("distance > 100",
        SqlPrinter.print(new Predicate("distance", Comparison.GREATER_THAN, 100L)));
    assertEquals("city = 'Odense'",
        SqlPrinter.print(new Predicate("city", Comparison.EQUALS, "Odense")));
}
```

When tests write files, use @TempDir

- @TempDir injects a fresh directory and deletes it afterwards

```
@Test
void keepsOnlyThePartitionsThatCanHoldAMatch(@TempDir Path directory) {
    StorageEngine engine = new StorageEngine(directory, maxRowsPerPartition: 2);
    engine.createTable(tableName: "trips", TRIPS);
    engine.copyFile(tableName: "trips", csv(fileName: "trips_sorted.csv"));

    Planner planner = new Planner(engine);
    Plan plan = planner.plan(select(new Predicate(columnName: "distance", Comparison.GREATER_THAN, constant: 200L)));

    assertEquals(new ScanStats(partitionsTotal: 4, partitionsRead: 1, partitionsPr... 3), plan.scanStats());
    assertEquals(List.of(210L, 299L), distances(plan));
}
```

Integration Tests

Run end-to-end tests against public API (e.g., SQL)

```
@Test
void runsASingleStatementAgainstTheTablesOfAnEarlierRun(@TempDir Path directory) throws IOException {
    runScript(directory, CREATE_AND_COPY);
    out.reset();

    // No ';' : a statement typed on the command line gets one, as it would in the DuckDB CLI
    int status = run(directory, ...args: "-c", "SELECT * FROM trips WHERE city = 'Esbjerg'");

    assertEquals(0, status);
    assertEquals("Esbjerg,299,450.25\n", out.toString(UTF_8));
}
```

Unit Tests vs. Integration Tests

Differences

	Unit Test	Integration Test
Calls	One Java method in isolation	The public API, end to end
Example	<code>printPredicateColumnOperator Constant()</code>	<code>"SELECT * FROM trips WHERE city = 'Esbjerg'"</code>
Naming	<code>*Test.java</code>	<code>*IT.java</code>
Runner	Surefire, during mvn test	Failsafe, during mvn verify

When finding a bug

- Reproduce in an integration test
- Fix the bug and expand unit tests
- Commit the tests & fix together

Agenda

Welcome

- Course overview
- AI policy & working with AI
- How much work is this?
- Architecture

Coffee break: Let's form teams

Workflow

- The engineering toolchain
- **The GitHub lifecycle**
- Logging from day one
- Dogfooding as design philosophy

This week's exercise and reading

Each team will use GitHub as code repository

- **Serves as documentation**
→ We will look at your code!
- **Serves as your collaboration platform**
→ Team members approve each other's changes
- **Serves as a continuous integration (CI) pipeline**
→ You can only merge when all tests pass
- **Serves as a to-do list**
→ Use GitHub Issues to track features and bugs



Share your repository with **alqeren1** (TA)

(part of Exercise 1 today)



GitHub as a Collaboration Platform

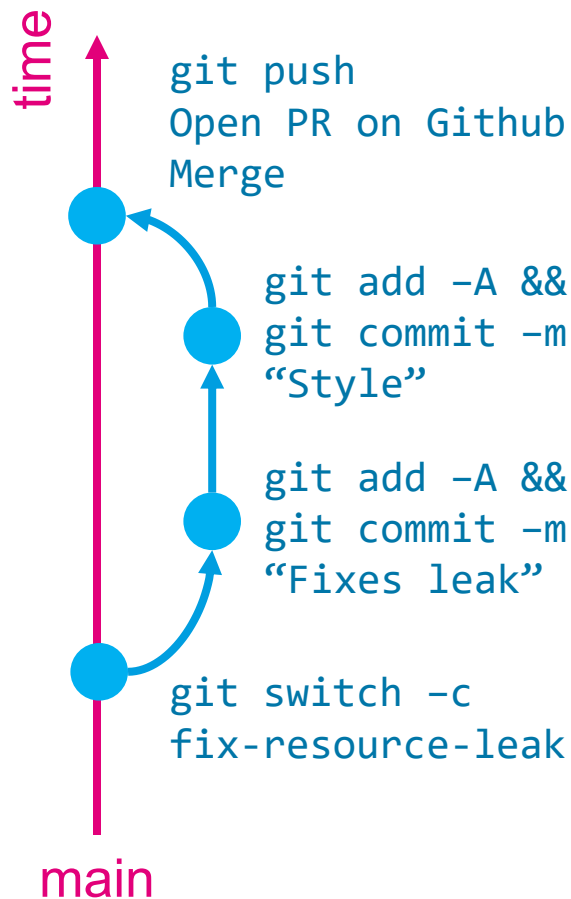
Protect the **main** branch

Require a pull request + 1 approval

- All changes to main go through a pull request (PR)
- Requires short-lived feature branches
- Request at least 1 approving review to merge feature branch into **main**
- Team members must read and approve each other's code

Tip: Configure automatic upstream setup

```
git config --global push.autoSetupRemote true
```





Demo

(create a feature branch, fix some code, commit, push, open PR, merge)



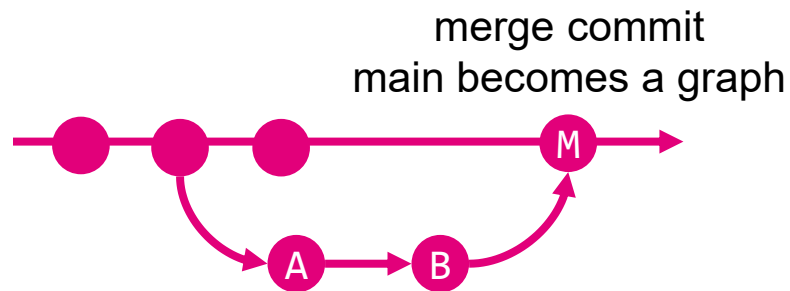
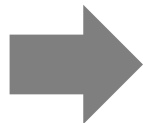
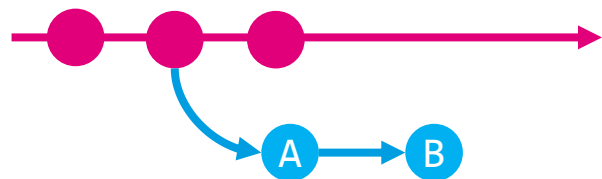
Protect the **main** branch, ctd.

- Require a pull request + 1 approval (previous slides)
- Block force pushes and deletions: main's history is append-only, a mistake is undone by a revert commit through a PR
- Do not allow bypassing by admins
- Require status checks to pass (see CI slides below)

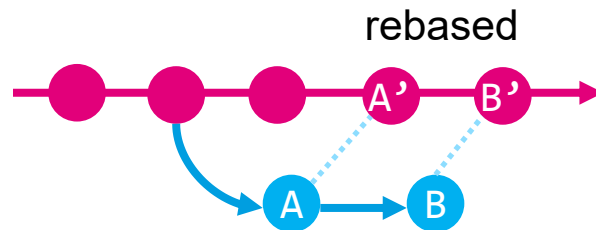
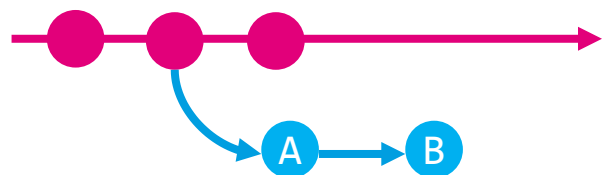


GitHub as a Collaboration Platform

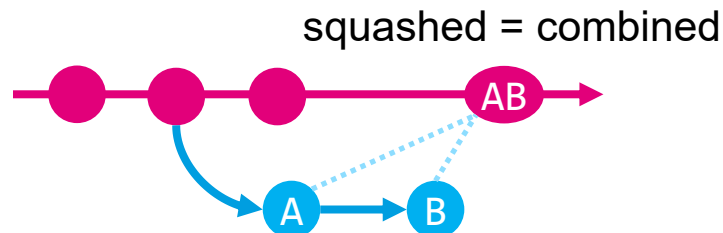
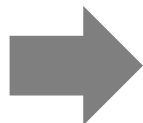
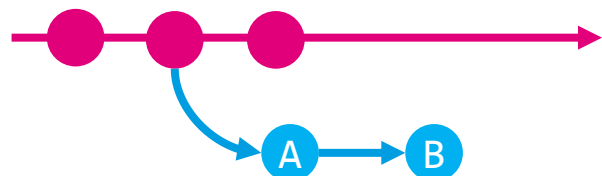
Merge commit



Rebase



Squash





GitHub as a Collaboration Platform

Comparison

Method	Linear history?	Keeps individual commits?	Adds merge commit?	Commits/PR on main
merge	No	Yes	Yes	many + 1
rebase	Yes	Yes	No	many
squash	Yes	No (combined)	No	1

Which to pick

- **squash: keeps main clean, messy WIP commits stay on the branch**
- merge: records exactly when each PR merged, merge bubbles in the history
- rebase: linear history that preserves individual commits

Tip: on GitHub, restrict allowed methods in Settings → General → Pull Requests (check only "Allow squash merging").



GitHub as a Continuous Integration Pipeline

Continuous Integration (CI)

- Create a CI workflow that produces status checks
- A status check is just the pass/fail result reported by a GitHub Actions job
- Add a workflow file, e.g. `.github/workflows/ci.yml`

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '25'
          cache: maven
      - run: mvn -B verify
```

What is -B again?



GitHub as a To-Do List

Ticketing systems, every company has them...

- GitHub Issues, JIRA, Linear, Azure Boards – all the same
- Since we use GitHub, we can use GitHub Issues
- How it works:
 1. Create an issue for every feature and bug
 2. Mention the issue number in the (squashed) commit message, using # (e.g., #1)
 3. Close the issue when done

Tip: GitHub scans PR messages and commit messages for specific keywords (like `fixes`, `closes`, or `resolves`) followed by an issue number. Tickets are closed automatically when the code is merged into the repository's main branch.

Avoid the following to enter version control using .gitignore:

- Build output (target/)
- Log files
- Secrets
- Large binaries
- IDE noise (.vscode/)

```
1  .vscode/
2
3  # Compiled class file
4  *.class
5
6  # Log file
7  *.log
8
9  # Package Files #
10 *.jar
11 *.war
12 *.nar
13 *.ear
14 *.zip
15 *.tar.gz
16 *.rar
17
18 # virtual machine crash logs, see http://www.java.com/en/download/help/error\_ho
19 hs_err_pid*
20 replay_pid*
21
22 # Hidden folders needed for designing the lecture slides and outline
23 samples/
24 screenshots/
25
26 # Maven build output
27 target/
28
29 # Written next to the POM by the shade plugin (Exercise 5 onwards)
30 dependency-reduced-pom.xml
31
32 # Engine log files (Exercise 1 onwards)
33 logs/
34
35 # Catalog and data files of the front door (Exercise 4 onwards)
36 data/
37
38 # Generated by bench/run.py when it runs in the repository (Exercise 6 onwards)
39 csv/
40 results/
41
42 # Scratch generator for the skewed join tables, kept out of the students' repos
43 bench/skew_tables.py
44
45 # Written by Python when bench/join_run.py imports bench/join_generate.py (Exer
46 __pycache__/
```

Agenda

Welcome

- Course overview
- AI policy & working with AI
- How much work is this?
- Architecture

Coffee break: Let's form teams

Workflow

- The engineering toolchain
- The GitHub lifecycle
- Logging from day one
- Dogfooding as design philosophy

This week's exercise and reading

Why Logging is (Extremely) Useful

Logging

- The record of what actually happened, otherwise your program is a black box
- Catches bugs and allows to build a reproducible test case
- Shows *decisions taken*, not just results, intermediate program states become log lines
- Your profiler: every operator may log durationMs= and rowsOut=

```
2026-08-07 20:43:19.195,3101dab2-2ec5-4eb8-ba06-67108cf27e80,1,3,DEBUG,Planner,table=customers column=c_city comparison=EQUALS const=city042 partition=0 min=city000 max=city099 decision=READ
2026-08-07 20:44:55.642,3101dab2-2ec5-4eb8-ba06-67108cf27e80,1,3,DEBUG,ScanOperator,table=customers version=1 partitions=1 rowsOut=1000
```

- And the log is *data*: our engine can ingest the log with COPY

How Logging is Set Up

SLF4J (API) + Log4j2 (backend)

One logger per class; only debug and error; {} placeholders:

```
private static final Logger LOGGER = LoggerFactory.getLogger(StorageEngine.class);
LOGGER.debug("table={} rows={} durationMs={}", tableName, rows, durationMs);
```

Context via MDC: sessionId (per run), statementNumber (per statement):

```
MDC.put("sessionId", UUID.randomUUID().toString());
MDC.put("statementNumber", String.valueOf(statementNumber));
```

Pattern layout renders the CSV schema (log4j2.xml):

```
<PatternLayout pattern="%d{...},%X{sessionId},%X{statementNumber},%tid,%level,%logger{1},%m%n"/>
```

Suggested columns: timestamp, sessionId, statementNumber, threadId, level, className, message

How to Read Logs

Grep

```
grep ,ERROR, logs/engine.log      # all errors
grep ,Planner, logs/engine.log    # all prune decisions
grep 3101dab2 logs/engine.log     # one session
```

Your engine!

```
CREATE TABLE logs (timestamp STRING, sessionId STRING, statementNumber LONG,
    threadId LONG, logLevel STRING, className STRING, logMessage STRING);
COPY logs FROM 'logs/engine.log';
SELECT * FROM logs WHERE statementNumber = 7; -- one statement
SELECT * FROM logs WHERE statementNumber > 0; -- all statements, minus setup
```

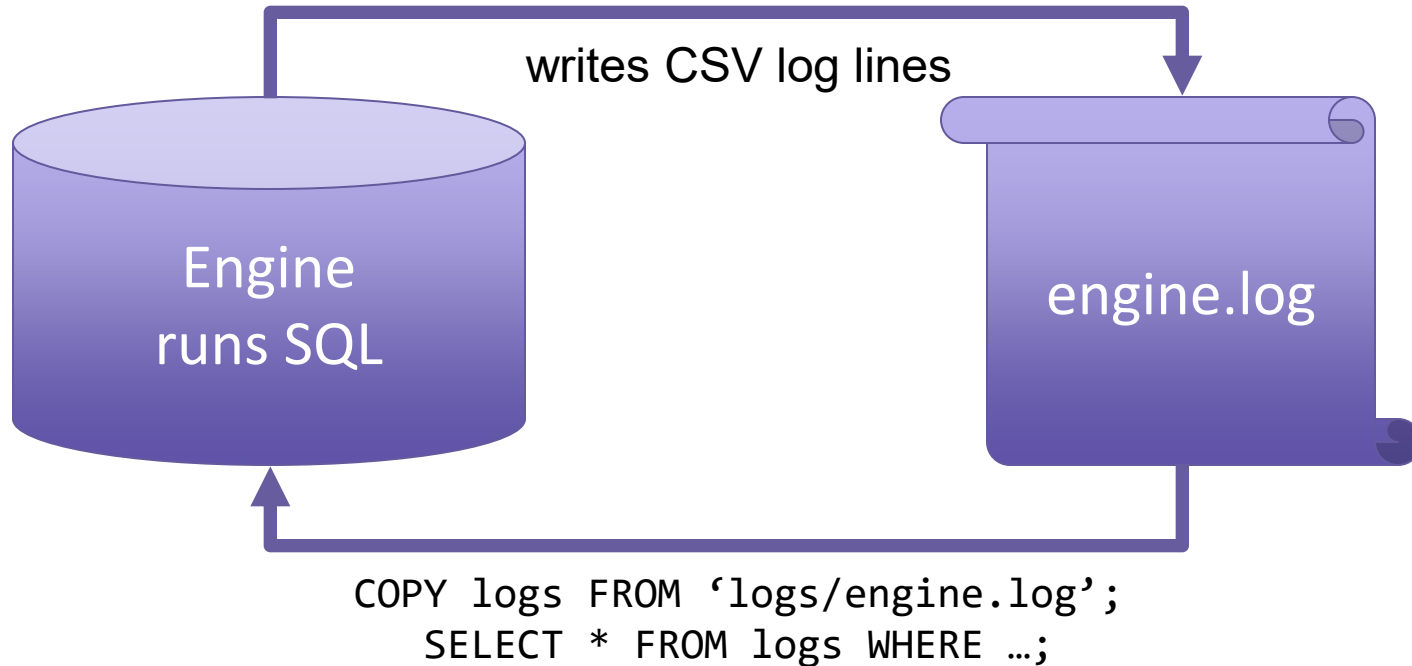
Project 1: load the log once, filter it

Project 2: re-ingest rotated log files with repeated COPY, retire old lines with DELETE

Project 3: join logs with dimension tables to analyze your own optimizer experiments

Dogfooding

“Eating your own dog food”
= using your tool to analyze/debug/monitor your tool



Dogfooding in the Industry

At Snowflake, all of this is made possible by Snowhouse - a Snowflake account that we use internally and has identical functionality to what our customers use. Among the vast number of data sets Snowhouse contains, it also captures usage telemetry. Each of us uses Snowhouse, from analyzing query performance to preparing data for quarterly roll-ups. The result is engineering's deep familiarity with the actual product we've built, which is very powerful and unique in our industry. It allows us to improve our product faster, feeding first-hand experience into designs.

Source: [The Rocket Behind Snowflake's Rocketship](#)

Agenda

Welcome

- Course overview
- AI policy & working with AI
- How much work is this?
- Architecture

Coffee break: Let's form teams

Workflow

- The engineering toolchain
- The GitHub lifecycle
- Logging from day one
- Dogfooding as design philosophy

This week's exercise and reading

Exercises & Readings

We publish the exercise sheets as PDFs and as MD files

- You can copy/paste the MD file into your repo and let AI implement it
- Your goal is to understand every line of code and be able to explain it
- We will stay here to help answer high-level questions, use AI to debug and explain low-level questions

Starting next week: We will ask teams to give a 5-minute overview of the state of their project + another team to give a 5-minute review of the reading material.

This Week: Exercise 1 & Reading

Exercise 1

- Form teams
- Tooling check: Java 25, Maven 3.9, Git, an IDE, a GitHub account
- Create your repository
- Maven project that builds, tests, and runs your engine
- SLF4J + Log4j2 writing the CSV log schema with rotation
- First unit test on `teamName()`
- CI on every PR, branch protection on main

Reading

- Hellerstein, Stonebraker & Hamilton, Architecture of a Database System (2007). Sections 1 and 4 required, skim the rest

Welcome again

Let's build a cool engine!

Java, Maven, JUnit, GitHub, Logging are fixed.

Understand decision tradeoffs:

sealed interfaces
vs. visitors

unit tests vs.
integration tests

merge commits vs.
rebase vs. squash

log details vs.
logging size